

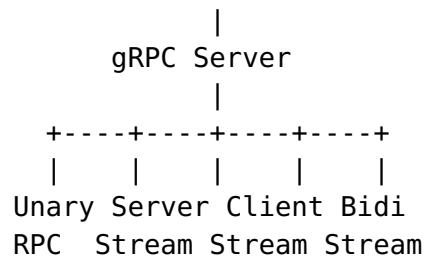
grpc-adapter

gRPC Adapter

The gRPC adapter defines a GRPCAdapter interface and provides a CLI implementation (GRPCCLIAdapter) that wraps the `grpcurl` command-line tool. It supports service discovery, unary and streaming method invocation, health checking, and TLS configuration.

Architecture

Go Challenge --> GRPCCLIAdapter --> `grpcurl [flags] <server> <method>`



The adapter shells out to `grpcurl` for each operation. `grpcurl` handles serialization, TLS, and protocol details. Responses are captured from `stdout` and parsed by the adapter.

Server Reflection

By default, the adapter uses gRPC server reflection to discover services and methods. This requires the server to register the reflection service:

```
import "google.golang.org/grpc/reflection"
reflection.Register(grpcServer)
```

For servers without reflection, use `WithProtoFiles()` to provide .proto file paths.

GRPCAdapter Interface

Defined in adapter_grpc.go:

```
type GRPCAdapter interface {
    ListServices(ctx context.Context) ([]string, error)
    ListMethods(ctx context.Context, service string) ([]string,
        error)
    Invoke(ctx context.Context, method, request string) (string,
        error)
    InvokeStream(ctx context.Context, method, request string)
        ([]string, error)
    HealthCheck(ctx context.Context, service string) (bool, error)
    Available(ctx context.Context) bool
    Close(ctx context.Context) error
}
```

Method Summary

Method	Description
ListServices	Returns all gRPC services via server reflection
ListMethods	Returns all methods for a given service
Invoke	Calls a unary gRPC method with JSON request, returns JSON response
InvokeStream	Calls a server-streaming method, returns all JSON responses
HealthCheck	Performs standard grpc.health.v1.Health/Check
Available	Returns true if the server is reachable (attempts ListServices)
Close	No-op (grpcurl does not maintain persistent connections)

GRPCOption Reference

```
type GRPCOption func(*grpcCLIConfig)
```

Option	Description	Default
WithTLS()	Enable TLS for connections	Disabled (plaintext)
WithCert(certFile)	Set TLS CA certificate file (also enables TLS)	(none)
WithInsecure()	Disable TLS certificate verification	false
WithProtoFiles(files...)	Use proto files instead of server reflection	(none)

Option	Description	Default
WithHeaders(map)	Add metadata headers to every request	(none)

Constructor

```
// Plaintext connection (default)
adapter := userflow.NewGRPCCLIAdapter("localhost:50051")

// TLS with custom cert
adapter := userflow.NewGRPCCLIAdapter(
    "api.example.com:443",
    userflow.WithCert("/path/to/ca.pem"),
)

// With auth header and proto files
adapter := userflow.NewGRPCCLIAdapter(
    "localhost:50051",
    userflow.WithInsecure(),
    userflow.WithHeaders(map[string]string{
        "Authorization": "Bearer <token>",
    }),
    userflow.WithProtoFiles("api/v1/service.proto"),
)
```

API Reference

ListServices

```
services, err := adapter.ListServices(ctx)
// ["grpc.health.v1.Health", "myapp.v1.UserService", ...]
```

Runs: `grpcurl -plaintext localhost:50051 list`

ListMethods

```
methods, err := adapter.ListMethods(ctx, "myapp.v1.UserService")
// ["myapp.v1.UserService/CreateUser", "myapp.v1.UserService/
    GetUser", ...]
```

Runs: `grpcurl -plaintext localhost:50051 list myapp.v1.UserService`

Invoke (Unary RPC)

```
response, err := adapter.Invoke(
    ctx,
    "myapp.v1.UserService/GetUser",
```

```

        `{"id": "user-123"}`,
    )
    // response:
        `{"id":"user-123","name":"Alice","email":"alice@example.com"}`
    ,

```

Runs: `grpcurl -plaintext -d '{"id":"user-123"}' localhost:50051 myapp.v1.UserService/GetUser`

InvokeStream (Server Streaming)

```

responses, err := adapter.InvokeStream(
    ctx,
    "myapp.v1.EventService/StreamEvents",
    `{"topic": "orders"}`,
)
// responses: [{"event\":"created\","...}"}, {"event\":"
               \ "updated\","...}"}]

```

The adapter parses streaming output by tracking JSON brace depth. Each complete `{...}` object is returned as a separate string in the slice.

HealthCheck

```

serving, err := adapter.HealthCheck(ctx, "myapp.v1.UserService")
// serving: true if response contains "SERVING"

```

Invokes `grpc.health.v1.Health/Check` with `{"service": "myapp.v1.UserService"}`. Pass an empty string to check the overall server health.

Streaming Support

The `InvokeStream` method handles server-streaming RPCs. The `parseJSONObjects` function splits `grpcurl`'s streaming output into individual JSON objects by tracking brace nesting depth:

```

Input:  {"event":"a"}{"event":"b"}{"event":"c"}
Output: [{"event":"a"}, {"event":"b"}, {"event":"c"}]

```

Client-streaming and bidirectional-streaming RPCs are not supported by the CLI adapter, as `grpcurl` has limited support for interactive streaming.

GRPCFlowChallenge

The `GRPCFlowChallenge` template executes a multi-step gRPC flow:

```

challenge := userflow.NewGRPCFlowChallenge(
    "CH-GRPC-001",
    "gRPC User Service Flow",
    "Verify CRUD operations on the User service",
    nil,
    adapter,
    userflow.GRPCFlow{
        ServerAddr: "localhost:50051",
        Steps: []userflow.GRPCStep{
            {
                Name:      "create-user",
                Method:    "myapp.v1.UserService/CreateUser",
                Request:    `{"name":"Alice","email":"alice@example.com"}`,
                ExpectedFields: map[string]interface{}{
                    "id": nil, // Field must exist (any value)
                },
                ExtractTo: map[string]string{
                    "id": "user_id",
                },
            },
            {
                Name:      "get-user",
                Method:    "myapp.v1.UserService/GetUser",
                Request:    `{"id":"{{user_id}}"}`, // Variable
substitution
                ExpectedFields: map[string]interface{}{
                    "name": "Alice",
                    "email": "alice@example.com",
                },
            },
        },
    },
)

```

Flow Features

- **Variable substitution:** Use `{{var_name}}` in method paths and request bodies
- **Field validation:** ExpectedFields with `nil` value means "field must exist"; non-`nil` values are compared as strings
- **Value extraction:** ExtractTo maps response fields to variables for use in subsequent steps
- **Step assertions:** Custom assertions (`response_contains`, `not_empty`, `stream_count`)
- **Per-step metrics:** Duration tracked for each step
- **Progress reporting:** Each step reports progress to the liveness monitor

Source Files

- Interface + CLI adapter: pkg/userflow/adapter_grpc.go
- Challenge template: pkg/userflow/challenge_grpc_flow.go